Exploring Instruction-Level Parallelism in Modern Processors

Rohail Shaikh

School of Computer and Information Sciences, University of the Cumberlands

MSCS 531-A01: Computer Architecture and Design

Dr. Gary Perry

September 20, 2026

Exploring Instruction-Level Parallelism in Modern Processors

Part 1: Research Literature Review

Introduction

Instruction-level parallelism (ILP) overlaps independent instructions from one program so a processor completes more useful work per clock cycle. Pipelining provides the basic overlap; modern processors add branch prediction, register renaming, dynamic scheduling, speculation, multiple issue, and large instruction windows. These features improve single-thread performance, but they also consume power, area, and verification effort. This review traces ILP's development and limits, then synthesizes four recent peer-reviewed studies on instruction delivery, branch prediction, scheduling, and memory dependence prediction. The studies show a shift from simply enlarging structures toward selective, context-aware mechanisms.

Historical Development of ILP

Pipelining was the earliest widely used form of ILP. It divides instruction processing into stages so different instructions can occupy fetch, decode, execute, memory, and writeback at the same time. Hazards prevent the ideal of one completion per cycle: true dependences stall data flow, branches disrupt control flow, and resource conflicts restrict overlap. Early reduced instruction set designs used regular formats and compiler scheduling, while forwarding and basic branch techniques reduced stalls (Hennessy & Patterson, 2017). The lasting principle is simple: performance improves when independent work overlaps safely.

Superscalar processors went further by issuing multiple instructions per cycle. Dynamic scheduling lets a ready younger instruction proceed while an older one waits. Register renaming removes false name dependences, and reorder buffers support out-of-order completion with in-order retirement. Better branch prediction enables deeper speculation. Compiler-centered VLIW

and EPIC designs exposed parallelism before execution but were less flexible when latency or control flow changed at run time (Hennessy & Patterson, 2017). Power and diminishing ILP gains encouraged multicore designs, yet each core still relies on ILP for single-thread speed.

Core Concepts and Constraints

Modern processors detect ILP through cooperating structures. The front end predicts the next program counter and fetches ahead. Decode creates operations; register renaming maps architectural registers to physical ones and removes name dependences. An issue queue tracks readiness and selects work for functional units. Memory dependence prediction may let loads pass older stores, and the reorder buffer commits completed instructions in program order to preserve precise exceptions. Available ILP is therefore limited by both the program and the hardware window used to find it.

True read-after-write dependences are fundamental because a consumer needs its producer's value. Control dependences make the correct path uncertain, and a branch misprediction discards wrong-path work. Unknown memory addresses can delay loads behind older stores. Hardware resources also limit how many operations can be fetched, issued, or completed. Wider pipelines increase bypass, selection, predictor, and recovery costs, so four-wide issue rarely produces a fourfold speedup.

Performance Measures and Trade-Offs

ILP effectiveness is commonly measured with instructions per cycle (IPC), cycles per instruction (CPI), execution time, branch misprediction rate, and utilization of pipeline resources. IPC emphasizes throughput, while CPI expresses the average cycle cost of committed instructions. A wider processor may raise IPC without reducing the latency of every individual instruction. Likewise, a technique can improve IPC but consume more power or area than its

gain justifies. Branch accuracy is important because each wrong prediction wastes fetch, decode, and execution work. For an experimental comparison, the workload, clock frequency, cache hierarchy, compiler options, and instruction count must remain consistent so that a change in IPC can reasonably be connected to the ILP feature being tested.

Memory System as an ILP Constraint

The memory hierarchy directly constrains ILP. Small L1 caches provide the lowest latency, larger L2 and L3 caches trade speed for capacity, and DRAM supplies much greater capacity with substantially higher latency. An instruction-cache miss can starve the front end, while a data-cache miss can delay an entire dependent chain. Out-of-order execution hides part of this delay by issuing independent work, but the benefit is limited by the reorder buffer, load-store queue, cache miss-handling resources, and memory bandwidth. Multiple misses overlap only when addresses and dependences permit memory-level parallelism. Prefetching and nonblocking caches can reduce waiting, but inaccurate prefetches consume bandwidth and energy. A bad memory-dependence prediction either replays an unsafe load or delays a safe one. Modern designs therefore coordinate scheduling, dependence prediction, translation, caches, and main memory instead of treating the memory hierarchy as separate from ILP.

Contemporary Research

Control Flow and Instruction Delivery

Khan et al. (2022) examined hard-to-predict branches in large data-center applications. Their Whisper design uses profile information to describe recurring branch behavior and supplements the hardware predictor for selected branches. The important contribution is not simply a larger predictor. It is a hybrid hardware-software approach that focuses on branches that repeatedly defeat conventional prediction. Whisper demonstrates that workload knowledge can

improve control-flow prediction without forcing every branch to use the most expensive mechanism. Its limitation is that profile-guided behavior may change when software versions or inputs change, so deployment requires an updated and representative profile.

Singh et al. (2024) addressed the refill delay that remains when a hard branch is mispredicted. Their alternate-path micro-operation cache prefetcher identifies low-confidence branches and selectively fetches decoded operations from the path not currently predicted. If the branch is wrong, those operations are already available for a faster pipeline refill. The authors reported a geometric-mean IPC gain of about 2% with modest storage overhead. The result is smaller than dramatic headline speedups, but it is meaningful because the design targets a specific high-cost event and avoids allocating full back-end resources to both paths. This study supports a broader trend toward selective assistance rather than unrestricted multipath execution.

Scheduling and Memory Dependence

Chen et al. (2023) proposed Orinoco to reduce the cost of conventional instruction scheduling and in-order commit. Orinoco uses ordered issue with unordered commit and non-collapsible queues. The design aims to preserve performance while avoiding some of the complex movement and selection logic found in large out-of-order structures. The reported average IPC improvement of 14.8% over the study's in-order-commit baseline shows that commit organization can become a meaningful bottleneck, not merely the final administrative stage of a pipeline. However, the result is relative to a particular modeled baseline and queue design, so it should not be interpreted as a universal gain for every processor.

Kim and Ros (2024) focused on memory dependence prediction, which decides whether a load should wait for an unresolved older store. Their PHAST predictor records the execution path between a conflicting store and load and chooses a history length appropriate for that load.

In their evaluation, a 14.5 KB design reduced memory-dependence mispredictions and came within 1.5% of an ideal predictor. PHAST illustrates why context matters more as processor windows and widths grow: a false dependence wastes available ILP through unnecessary waiting, while a missed dependence causes replay and lost work. The design also shows the continuing importance of balancing accuracy against predictor storage and lookup complexity.

Critical Synthesis and Future Directions

The four studies attack different points in the same flow of instructions. Whisper tries to select the correct path, alternate-path micro-operation prefetching reduces recovery time when that selection fails, Orinoco reorganizes scheduling and completion, and PHAST permits safe overlap among memory operations. Their shared lesson is that simply increasing pipeline width or queue size is no longer sufficient. Wider designs increase the cost of wrong predictions, dependence mistakes, and instruction-supply gaps. New gains therefore depend on mechanisms that identify when expensive action is likely to help.

Promising work includes tighter compiler-hardware cooperation, confidence-guided features, and predictors that adapt to phase changes without excessive storage. Machine learning may classify hard events, but it must satisfy strict latency, energy, and explainability limits. Heterogeneous systems can reserve wide out-of-order cores for latency-sensitive code while smaller cores and accelerators provide throughput. Because speculation can expose microarchitectural state, security must be designed alongside performance. Future processors will likely combine moderate width with accurate prediction, fast recovery, and specialized resources rather than pursue unlimited ILP.

Part 2: Practical Exploration With gem5

Purpose and Experimental Design

The practical experiment uses gem5 syscall-emulation mode and one x86 out-of-order CPU. This mode runs a user program without booting a complete operating system, which keeps the assignment manageable. A width-one O3CPU provides the single-issue baseline, and a width-four O3CPU represents a superscalar design. The configuration uses a 2 GHz clock, separate 32 KiB instruction and data caches, a 256 KiB L2 cache, and 512 MiB of DDR3 memory. One Hello World program validates execution. Three small programs emphasize independent integer operations, independent floating-point operations, and branch-plus-memory behavior. The O3 pipeline trace shows fetch, decode, rename, dispatch, issue, completion, and retirement events (The gem5 Project, 2026a).

Repository: https://github.com/rohailshaikh/MSCS_531_Assignment_4_Rohail_Shaikh

Table 1

Simulated Processor Configuration

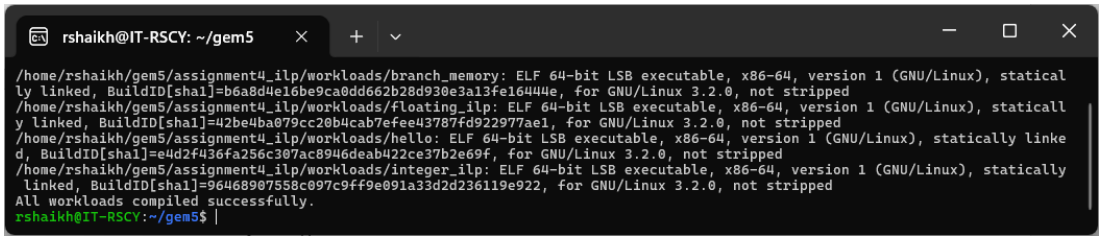
Component	Setting
ISA and mode	x86-64 syscall emulation
CPU	X86O3CPU, one core
Clock	2 GHz
Single issue	Fetch/decode/rename/dispatch/issue/commit width = 1
Superscalar	The same controlled widths = 4
Branch comparison	Tiny 1-bit LocalBP vs. standard TournamentBP conditional component
SMT	Two software threads on one O3CPU core
Memory	32 KiB L1I + 32 KiB L1D, 256 KiB L2, 512 MiB DDR3

Implementation Evidence

Figure 1 documents the package location and successful compilation of the statically linked x86-64 workloads. Static linking reduces dependency problems in syscall-emulation mode. Figure 2 records the O3CPU width and predictor settings used in the comparison.

Figure 1

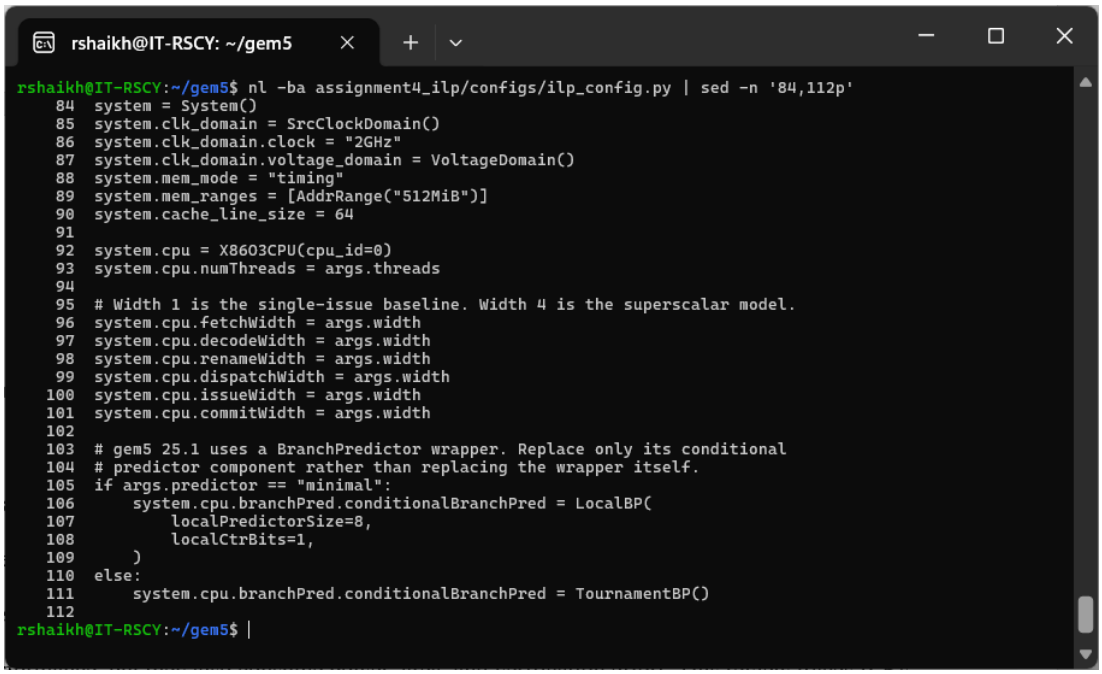
Successful Static Workload Compilation



Note. The terminal confirms that all four x86-64 workloads were compiled as statically linked executables.

Figure 2

O3CPU Width and Branch Predictor Configuration



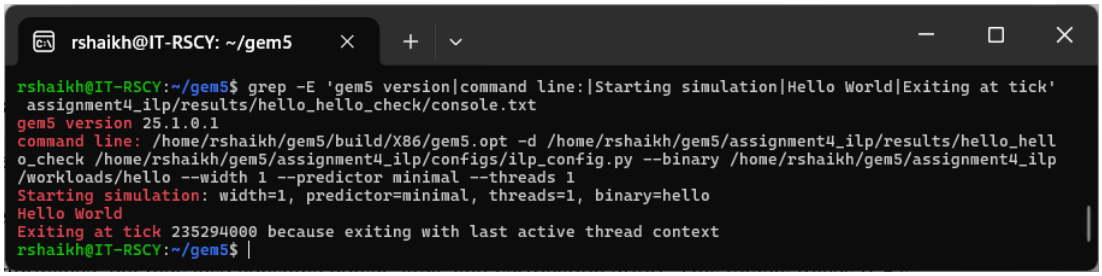
Note. The configuration shows the 2 GHz clock, 512 MiB memory range, width controls, and conditional LocalBP and TournamentBP selections.

The first simulation confirms that the environment, binary format, and configuration work before the longer experiments begin. Figure 3 shows the gem5 command, the expected

Hello World output, and the normal exit message. This evidence directly addresses the implementation criterion in the rubric.

Figure 3

Successful Hello World Execution in gem5



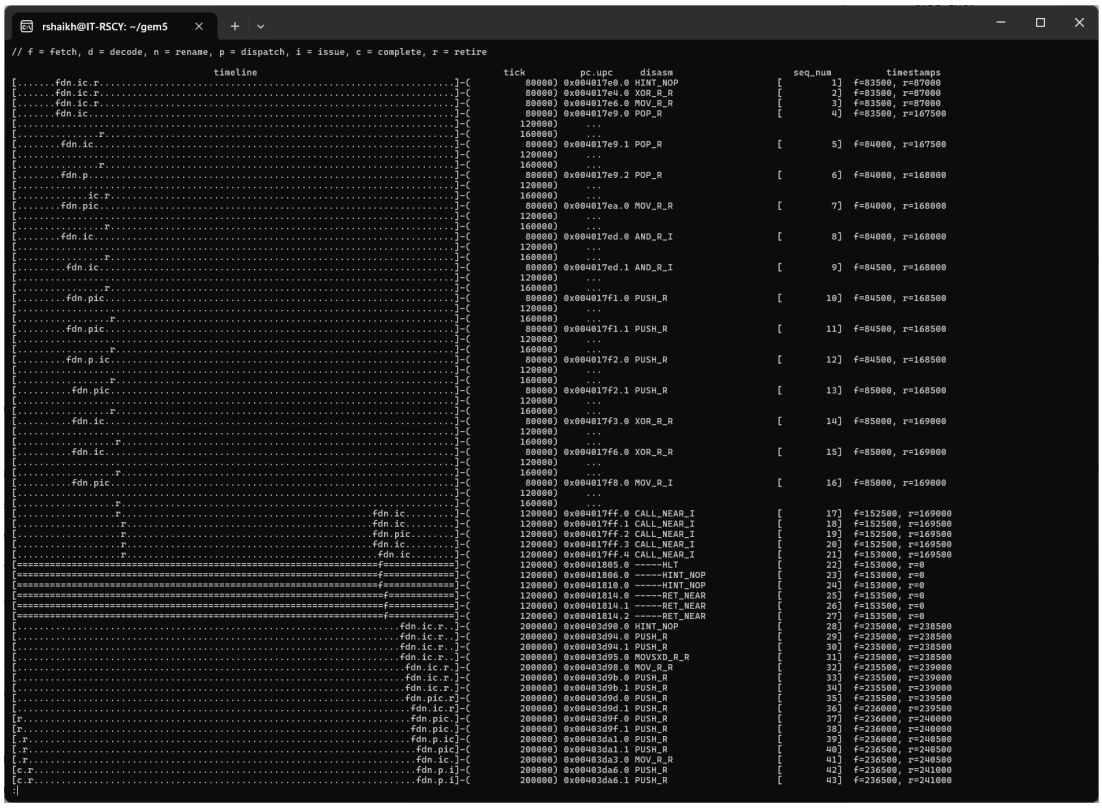
Note. The terminal output confirms gem5 version 25.1.0.1, the simulation configuration, Hello World, and a normal exit.

Pipeline Observation and Latency

The O3PipeView trace is a text-based pipeline visualization. Its symbols show fetch, decode, rename, dispatch, issue, completion, and retirement. Overlap among rows demonstrates that several instructions are in flight at the same time. A long horizontal gap before issue or retirement indicates waiting caused by a dependence, resource conflict, cache access, or earlier instruction. Fetch-to-retire latency is measured from the timestamps printed for a sample of committed instructions. This latency should not be confused with CPI: latency follows an individual instruction through the pipeline, while CPI represents average system throughput.

Figure 4

O3 Pipeline Flow for the Branch-Memory Workload



Note. The stage legend and overlapping instruction rows show fetch, decode, rename, dispatch, issue, completion, and retirement activity.

Table 2

Pipeline Latency From the O3PipeView Sample

Measure	Cycles
Average fetch-to-retire latency	40.71
Median fetch-to-retire latency	9.00
Minimum latency	7.00
Maximum latency	168.00

Measured Performance

IPC is calculated as committed instructions divided by processor cycles. CPI is cycles divided by committed instructions. The same binaries, clock, and cache hierarchy were used for each comparison. In gem5 25.1.0.1, the O3CPU branch predictor is a wrapper containing a conditional predictor. The minimal case uses a deliberately tiny one-bit LocalBP conditional

component as a weak baseline, while the comparison case uses the standard TournamentBP component.

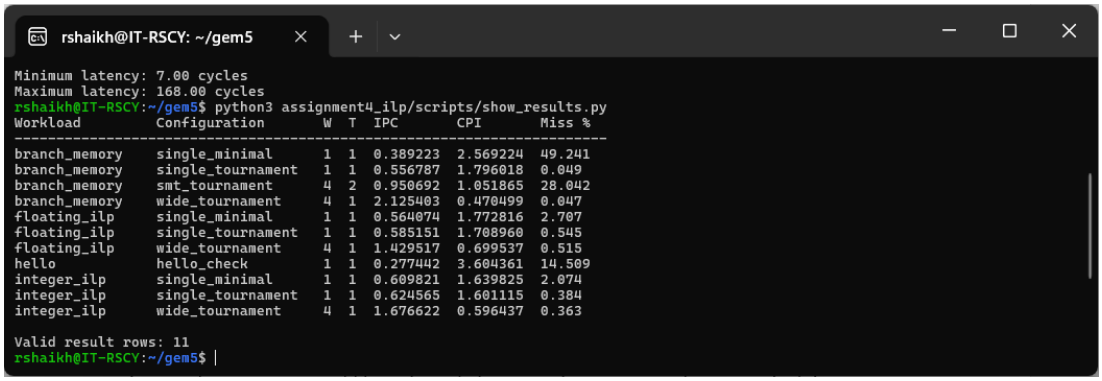
Table 3

Selected Measured gem5 Performance Results

Workload	Configuration	IPC	CPI	Branch miss %
Integer	Single/tournament	0.624565	1.601115	0.384
Integer	Wide/tournament	1.676622	0.596437	0.363
Floating point	Single/tournament	0.585151	1.708960	0.545
Floating point	Wide/tournament	1.429517	0.699537	0.515
Branch-memory	Single/minimal	0.389223	2.569224	49.241
Branch-memory	Single/tournament	0.556787	1.796018	0.049
Branch-memory	Wide/tournament	2.125403	0.470499	0.047
Branch-memory	SMT/tournament	0.950692	1.051865	28.042

Figure 5

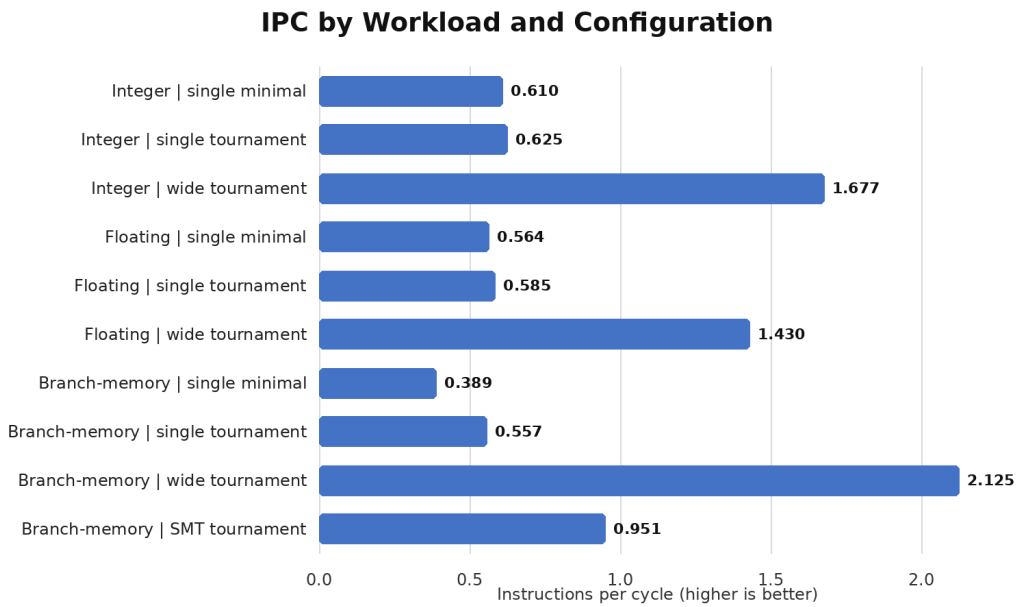
Collected gem5 Statistics



Note. The output contains 11 valid result rows and reports IPC, CPI, and branch-misprediction rate for each measured configuration.

Figure 6

Instructions per Cycle by Workload and Configuration



Note. IPC values were calculated from the measured gem5 instruction and cycle counts.

Analysis of ILP Techniques

Multiple issue. With TournamentBP held constant, widening the controlled pipeline stages from one to four increased integer IPC from 0.624565 to 1.676622, a 168.4% improvement. Floating-point IPC increased from 0.585151 to 1.429517, a 144.3% improvement. The independent accumulators gave the O3CPU more ready work to issue at the same time. The gains remained below an ideal fourfold increase because dependencies, pipeline overhead, cache activity, and finite functional units still limited parallel execution.

Branch prediction. At width one, TournamentBP reduced the branch-memory misprediction rate from 49.241% with the minimal LocalBP to 0.049%. IPC increased from 0.389223 to 0.556787, which is a 43.1% improvement. The more accurate predictor prevented repeated wrong-path work and pipeline recovery. The unusually large accuracy difference reflects the intentionally weak baseline and predictable synthetic branch pattern, so it should not be treated as a universal hardware improvement.

SMT. The two-thread branch-memory run produced a total IPC of 0.950692, compared with 2.125403 for the one-thread width-four run. This was a 55.3% decrease in aggregate IPC, even though the SMT run completed twice as many instructions. The two identical branch-heavy threads competed for shared fetch bandwidth, queues, functional units, cache capacity, and prediction state; the SMT misprediction rate also rose to 28.042%. This result shows that SMT helps only when additional threads can use otherwise idle resources without creating greater contention.

Troubleshooting and Validation

No major execution error occurred. Before simulation, each workload was compiled with static linking and checked with the file command to confirm an x86-64 statically linked executable. This validation reduced the risk of dynamic-loader or library failures in syscall-emulation mode. The Hello World output, deterministic workload results, generated statistics files, and normal exit messages confirmed that the configuration and binaries executed correctly.

Conclusion

ILP remains essential for single-thread performance, but its value depends on independence and the cost of finding it. Current research favors targeted prediction, recovery, and scheduling. In gem5, issue width changes how many ready instructions begin together, branch prediction affects wasted work, and SMT can use idle resources while adding contention. These results describe a controlled educational model, not commercial processor performance.

References

Chen, D., Zhang, T., Huang, Y., Zhu, J., Liu, Y., Gou, P., Feng, C., Li, B., Wei, S., & Liu, L. (2023). Orinoco: Ordered issue and unordered commit with non-collapsible queues. In Proceedings of the 50th Annual International Symposium on Computer Architecture (Article 11, pp. 1-14). Association for Computing Machinery. https://doi.org/10.1145/3579371.3589046

Hennessy, J. L., & Patterson, D. A. (2017). Computer architecture: A quantitative approach (6th ed.). Morgan Kaufmann.

Khan, T. A., Ugur, M., Nathella, K., Sunwoo, D., Litz, H., Jimenez, D. A., & Kasikci, B. (2022). Whisper: Profile-guided branch misprediction elimination for data center applications. In 2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO) (pp. 19-34). IEEE. https://doi.org/10.1109/MICRO56248.2022.00017

Kim, S. S., & Ros, A. (2024). Effective context-sensitive memory dependence prediction. In 2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA) (pp. 515-527). IEEE. https://doi.org/10.1109/HPCA57654.2024.00045

Singh, S., Perais, A., Jimborean, A., & Ros, A. (2024). Alternate path micro-op cache prefetching. In 2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA) (pp. 1230-1245). IEEE. https://doi.org/10.1109/ISCA59077.2024.00092

The gem5 Project. (2026a). Visualization. https://www.gem5.org/documentation/general_docs/cpu_models/visualization/

The gem5 Project. (2026b). Building gem5. https://www.gem5.org/documentation/general_docs/building